

sends them to two display fields to be shown. If you pass a list to the `setValue()` method of a `DisplayText` object, it will display each element on a separate line. We use two `DisplayText` objects because `self.namesField` is left justified, while `self.scoresField` is right justified.

The `handleInputs()` method ❷ only needs to check for and respond to the user clicking the Quit, Back, and Reset Scores buttons. Because the Reset Scores button wipes data, we should ask for confirmation before performing this action. Therefore, when the user clicks this button, we call an intermediate function, `showCustomResetDialog()` ❸, to put up a dialog asking the user to confirm that they really want to clear out all the current scores.

The `draw()` method ❹ draws all the elements in the window.

Finally, the `respond()` method ❺ allows another scene to ask this scene for information. This is what allows the Play scene to request the highest current score and the tenth-highest score—the minimum score to qualify a player for the high scores list. The caller sends a value that indicates what information it’s looking for. In this case, the requested info is `HIGH_SCORES_DATA`, a constant that is shared from the *Constants.py* file. This method builds up a dictionary of the two requested values and returns it to the calling scene.

File: Dodger/HighScoresData.py

The final class is `HighScoresData`, responsible for managing the high score information. It reads and writes the data as a file in JSON format. The data is always kept in order, from the highest to the lowest score. For example, the file representing the 10 highest scores might look like this:

```
[['Moe', 987], ['Larry', 812], ... ['Curly', 597]]
```

[*Listing 16-14*](#) shows the code of the `HighScoresData` class.

```
# HighScoresData class
from Constants import *
from pathlib import Path
import json

class HighScoresData():
```